Java Silver（SE 17・1Z0-825-JPN）　3 演算子と制御構造　#05

演算子

組み方は優先順位で、計算は左から ― 割り算と余り、++ の前と後、複合代入、== の意味、短絡評価、三項演算子

各ページの下に、根拠にした Oracle の資料（JLS 15 章・Java Tutorials）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 3（演算子の使用（演算子の優先順位を含む））

1 算術演算子とインクリメント
整数の / は 0 方向に切り捨て。余りの符号は左。x++ は元の値

2 代入と比較
代入式は値を持つ。複合代入はキャストつき。== は値か参照か

3 論理演算子とビット演算子
&& と || は短絡する。& と | はしない。^ は XOR

4 優先順位と三項演算子
組み方は表で決まり、計算は左から。三項は右から組む

1 つの式を読む 2 つの決まり（Claude が作成）

決まり	何を決めるか
優先順位	どの演算子とどのオペランドを**組にするか**（かっこの付け方）
結合規則	同じ優先順位が並んだとき、**左右どちらから**組にするか
評価の順序	組ができたあと、**どの順に計算するか**。二項演算子は左のオペランドが先

```
int i = 1;
i = i++ + i++ * 2;    // 組：i++ + (i++ * 2)
                      // 計算：1 + (2 * 2) = 5
```

芯：優先順位は「組み方」だけ。* が先に組になっても、**左の i++ が先に計算**される。

出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JLS 15.7.1 Evaluate Left-Hand Operand First（The left-hand operand of a binary operator appears to be fully evaluated before any part of the right-hand operand is evaluated）／Java Tutorials「Operators」／結果は JDK 17 で確かめたもの

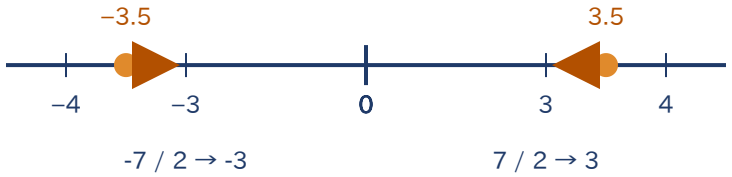
第1章

算術演算子とインクリメント

整数の割り算は 0 の方向へ。++ は前か後ろかで式の値が変わる。

割り算と余り

整数の / は 0 の方向に切り捨てる（Claude が作成）



どちらも 0 に近いほうへ。−4 にはならない

余りの決まり：(a / b) * b + (a % b) が a に戻る。だから余りの符号は**左（割られる数）**と同じ。

式	結果（JDK 17）
`-7 % 3`・`7 % -3`	−1・1
`7.5 % 2`	1.5（double にも使える）
`double d = 10 / 4;`	2.0（先に int で 2）
`10 / 4.0`	2.5
`5 / 0`・`5 % 0`	ArithmeticException: / by zero
`5.0 / 0`・`-5.0 / 0`	Infinity・−Infinity
`0.0 / 0`・`5 % 0.0`	NaN・NaN

0 割りで例外になるのは整数だけ。浮動小数点の割り算は例外を出さない。どちらもコンパイルは通る。

出典：JLS 15.17.2 Division Operator /（Integer division rounds toward 0／if the value of the divisor in an integer division is 0, then an ArithmeticException is thrown／evaluation of a floating-point division operator / never throws a run-time exception）・15.17.3 Remainder Operator %（(a/b)*b+(a%b) is equal to a）／結果は JDK 17 で確かめたもの

前置と後置 ー 式の値はどちらか

x = 5 のとき x++ + ++x を左から追う（Claude が作成）			
手順	部品	式の値	x の値
1	x++	5（元の値）	6
2	++x	7（増やした値）	7
3	5 + 7	12	7

解き方：表のように「式の値」と「x の今の値」を 2 列で書き出す。前置は先に変えて新しい値、後置は元の値を使ってから変える。

コード	結果（JDK 17）
`x = 5; x = x++;`	x は 5（元の値で上書き）
`a = 3; b = a-- - --a;`	b は 2、a は 1
`i = 1; i = i++ + i++ * 2;`	i は 5
`char c = 'A'; c++;`	'B'（型は char のまま）
`double d = 1.5; d++;`	2.5
`final int x = 1; x++;`	cannot assign a value to final variable
`5++`・`(a + b)++`	コンパイルエラー（変数でない）

出典：JLS 15.14.2 Postfix Increment Operator ++（The value of the postfix increment expression is the value of the variable before the new value is stored／A variable that is declared final cannot be incremented）・15.15.1 Prefix Increment Operator ++（The value of the prefix increment expression is the value of the variable after the new value is stored）／結果は JDK 17 で確かめたもの

評価の順序 ― 左のオペランドが先

a[i] = i = 2;（i は 0）の計算（Claude が作成）

① 左を決める
a[0]

② 右を計算
i = 2 → 2

③ 入れる
a[0] = 2

結果：a[0] が 2、a[2] は 0、i は 2

x += x -= 3;（x は 10）

左の x の値 10 を先に覚える → 右で x は 7 → 10 + 7 = 17

場面	先に計算されるもの
二項演算子	左のオペランドを全部。そのあと右
複合代入	左の変数を決め、**今の値を覚えて**から右
配列への代入	配列と添字を先に決めてから右
メソッドの引数	左の引数から順に
&&・||・? :	左（条件）だけで決まれば、右は計算しない

代入は右から組になるが、左辺の変数（配列の添字など）は先に決まる。
「右から計算」ではない。

出典：JLS 15.7.1（If the operator is a compound-assignment operator, then evaluation of the left-hand operand includes both remembering the variable … and fetching and saving that variable's value）・15.7.4 Argument Lists are Evaluated Left-to-Right・15.26.1（the array reference subexpression … then the index subexpression … are evaluated）／結果は JDK 17 で確かめたもの

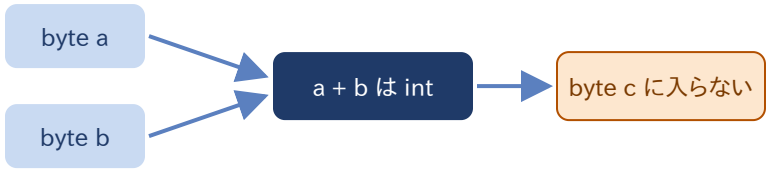
第2章

代入と比較

代入式も値を持つ。== は数なら値、参照ならオブジェクトを比べる。

代入・複合代入と型

byte どうしの + の型（Claude が作成）



int より小さい型は計算の前に int に昇格（#02）

コード	結果（JDK 17）
`byte c = a + b;`	possible lossy conversion
a・b が final で値 10・20	OK（定数式 30）
`byte f = 10 + 20;`	OK（定数式）

複合代入	結果（JDK 17）
`short s = 10; s += 5.7;`	15（(short)15.7）
`byte b = 10; b *= 30;`	44（(byte)300）
`char c = 'a'; c += 1;`	'b'
`int y = 2; y *= 3 + 4;`	14（右辺全体にかかる）
`s = s + 1;`（s は short）	possible lossy conversion

E1 op= E2 は E1 = (T)((E1) op (E2))。T は左辺の型。キャストが入るのでコンパイルは通るが、値は切り捨てや桁あふれで変わる。

代入式は値を持つ：`a = b = c = 7` は右から組み、3 つとも 7。

出典：JLS 15.26.2 Compound Assignment Operators（E1 op= E2 is equivalent to E1 = (T) ((E1) op (E2)), where T is the type of E1）・15.26（the result of the assignment expression is the value of the variable after the assignment has occurred）／JLS 5.2・5.6（the value of the constant expression is representable in the type of the variable）／結果は JDK 17 で確かめたもの

== で比べるもの

式	結果（JDK 17）
`10 == 10.0`	true（double にそろえる）
`'a' == 97`	true（文字コード）
`0.1 + 0.2 == 0.3`	false
`0.1 + 0.2` の表示	0.30000000000000004
`Double.NaN == Double.NaN`	false（NaN はどれとも等しくない）
`Integer a = 127, b = 127; a == b`	true（同じオブジェクト）
`Integer e = 1000; int f = 1000; e == f`	true（アンボクシングして値）
`e.equals(f)`	true（値で比べる）

−128〜127 の int をボクシングすると必ず同じオブジェクト（JLS 5.1.7）。それ以外は保証がない（JDK 17 の標準設定では Integer 128 どうしの == は false）。

オペランド	== の意味
数値どうし	昇格してから**値**を比べる
boolean どうし	値を比べる
参照どうし	**同じオブジェクトか**（中身は equals）
ラッパーと基本型	アンボクシングして値
int と boolean	コンパイルエラー

条件は boolean だけ（C 言語とのちがい）

```
if (x = 1)        // int → コンパイルエラー
boolean b = false;
if (b = true)     // 代入。いつも true で通る
```

出典：JLS 15.21.1（binary numeric promotion is performed on the operands／If either operand is NaN, then the result of == is false）・15.21.2・15.21.3 Reference Equality Operators／JLS 5.1.7 Boxing Conversion（an integer in the range -128 to 127 inclusive … It is always the case that a == b）／JLS 14.9（must have type boolean or Boolean）／結果は JDK 17 で確かめたもの

第3章

論理演算子とビット演算子

&& と || は左で決まれば右を見ない。& と | は必ず両方を見る。

短絡評価

右を計算するかどうか（Claude が作成）		
A && B	A が false	B は見ない → false
	A が true	B を計算
A || B	A が true	B は見ない → true
	A が false	B を計算

```
if (s != null && s.length() > 0)  // null でも安全
if (s != null & s.length() > 0)   // null なら NPE
```

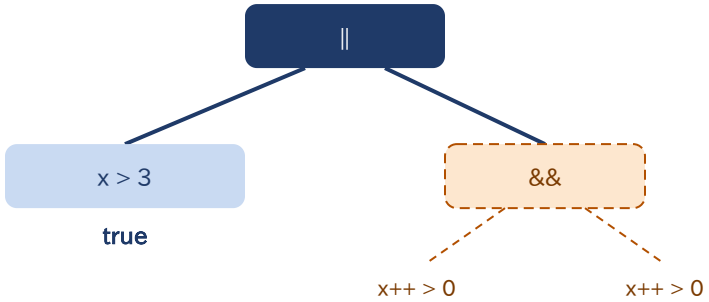
x = 0 から順に実行（JDK 17 の結果）		
文	結果	x
`r1 = (x > 0) && (++x > 0);`	false	0（右を見ない）
`r2 = (x > 0) & (++x > 0);`	false	1（右も計算）
`r3 = (x > 0) || (++x > 0);`	true	1（右を見ない）

演算子	true になるとき	短絡
&&・&	両方 true	&& だけ
||・|	どちらかが true	|| だけ
^	ちょうど一方が true	しない
!	false のとき	単項

出典：JLS 15.23 Conditional-And Operator &&（evaluates its right-hand operand only if the value of its left-hand operand is true）・15.24 Conditional-Or Operator ||（only if the value of its left-hand operand is false）・15.22.2 Boolean Logical Operators &, ^, and |（For ^, the result value is true if the operand values are different）／結果は JDK 17 で確かめたもの

&& は || より先に組になる

x > 3 || x++ > 0 && x++ > 0（x は 5）の組み方（Claude が作成）



式	組み方と結果（JDK 17）
`a || b && c`	`a || (b && c)`
`!a == b`	`(!a) == b`（a=true, b=false で true）
`!x > 5`（x は int）	bad operand type int for unary operator '!'
`!(x > 5)`	OK
`x > 1 == true`	`(x > 1) == true`（OK）
`a & b | c`	`(a & b) | c`

! は単項演算子なので強い。! の後ろに比較を書くときはかっこで囲む。

出典：Java Tutorials Operator Precedence（unary は relational・equality より上、logical AND は logical OR より上）／JLS 15.15.6（The type of the operand expression of the unary ! operator must be boolean or Boolean）・15.22（& having the highest precedence and | the lowest）／結果は JDK 17 で確かめたもの

ビット演算とシフト

6 と 3 を 2 進数で（下 3 けた、Claude が作成）				
	4	2	1	10 進
6	1	1	0	6
3	0	1	1	3
6 & 3（両方 1）	0	1	0	2
6 | 3（どちらか 1）	1	1	1	7
6 ^ 3（ちがえば 1）	1	0	1	5

~x は全ビット反転で −x − 1。~5 は −6。^ はべき乗ではない（2 ^ 3 は 1）。

シフト	意味	例（JDK 17）
<<	左へずらす（×2 ずつ）	8 << 2 → 32
>>	右へ。符号を保つ	-8 >> 1 → −4
>>>	右へ。左は 0 で埋める	8 >>> 1 → 4
>>>（負）	大きな正の数になる	-1 >>> 28 → 15

同じ記号が 2 役：&・|・^ は、オペランドが boolean なら論理演算子（短絡しない）、整数ならビット演算子。

出典：JLS 15.22.1 Integer Bitwise Operators &, ^, and |・15.15.5 Bitwise Complement Operator ~（~x equals (-x)-1）・15.19 Shift Operators（n >> s … with sign-extension／n >>> s … with zero-extension）／結果は JDK 17 で確かめたもの

第4章

優先順位と三項演算子

迷ったらかっこ。三項演算子は値を選ぶ「式」。

優先順位の表

順	種類	演算子
1	後置	`x++ x--`
2	単項	`++x --x +x -x ~ !`
3	乗除	`* / %`
4	加減	`+ -`
5	シフト	`<< >> >>>`
6	関係	`< > <= >= instanceof`
7	等価	`== !=`
8〜10	ビット・論理	`&` → `^` → `|`
11〜12	条件	`&&` → `||`
13	三項	`? :`
14	代入	`= += -= *= /= %=` など

式	結果（JDK 17）
`2 + 3 * 4 - 6 / 2 % 2`	13（2 + 12 − 1）
`10 - 4 - 3`	3（左から）
`2 * 3 % 4`	2（左から）
`x++ + x++ + " " + x`（x は 1）	"3 3"
`"" + x++ + x++`（x は 3）	"34"

> **結合の向き：**代入以外の二項演算子は**左から**、代入演算子と三項演算子は**右から**組になる。

出典：Java Tutorials「Operators」Operator Precedence（All binary operators except for the assignment operators are evaluated from left to right; assignment operators are evaluated right to left）／JLS 15.17・15.18（syntactically left-associative）・15.25（right-associative）／結果は JDK 17 で確かめたもの

三項演算子

```
int x = 7;
String s = x > 5 ? "big" : x > 2 ? "mid" : "small";
// x > 5 ? "big" : (x > 2 ? "mid" : "small") → big

int y = 0;
int z = true ? y++ : y--;   // y-- は計算しない
// y は 1、z は 0
```

NG（JDK 17 の表示）

```
x > 2 ? System.out.println("a") : System.out.println("b");
// not a statement
String t = x > 2 ? "ok" : 5;
// bad type in conditional expression
```

2 つの値の型	式全体の型（例と結果）
同じ型	その型
int と double	double：`true ? 1 : 2.0` → 1.0
char と char に入る int の定数	char：`false ? 'a' : 98` → b
String と int	Object などの参照型（String の変数には入らない）

式なので値が必要。単独の文にできず、void のメソッドも書けない。処理を分けるなら if 文。

選ばれなかった側は計算されない。++ や メソッド呼び出しがあっても実行されない。

出典：JLS 15.25 Conditional Operator ? :（The conditional operator is syntactically right-associative／It is a compile-time error for either the second or the third operand expression to be an invocation of a void method／The operand expression not chosen is not evaluated）・15.25.2 Numeric Conditional Expressions／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
割り算・余りの結果	整数どうしは 0 方向に切り捨て。余りの符号は左。0 割りは整数だけ例外
++ と -- がまざった式	左から「式の値」と「変数の値」を書き出す。後置は元の値。x = x++ は変わらない
複合代入の結果	左の値を先に覚える。左辺の型へのキャストつき。右辺全体にかかる
コンパイルできるか	byte + byte は int。条件は boolean だけ。! は boolean だけ。三項演算子は単独の文にならない
== の結果	数は昇格して値。参照は同じオブジェクトか。浮動小数点と NaN に注意
&&・|| と ++ の組合せ	左で決まれば右は実行されない。& と	は両方実行。&& が || より先に組む
三項演算子の表示	右から組む。式全体の型（1.0 や文字）に注意

出典：このPDFの各ページの資料

まとめ

算術とインクリメント
0 方向に切り捨て・余りは左の符号。後置は元の値

代入と比較
複合代入はキャストつき。== は値か参照か

論理とビット
&& と || だけ短絡。^ は XOR。~x は −x−1

優先順位と三項
組み方は表、計算は左から。三項は右から組む式

式	結果
`-7 / 2・-7 % 3`	−3・−1
`x++ + ++x`（x は 5）	12（x は 7）
`b *= 30`（byte b は 10）	44
`(0 > 0) && (++x > 0)`	false（x は増えない）
`true ? 1 : 2.0`	1.0

次は #06 分岐と switch。if 文、switch 文と switch 式（アロー・yield）へ進む。

出典：このPDFの各ページの資料